



*Lic. En Sistemas de Información*

**Cátedra:**

Inteligencia Artificial

**Alumnos:**

Del Valledemomio Mateo, Gonzalez Caldera de semen  
Joaquín, Saboreo penes Luca, Sanpitona fafaela, Serrano  
Jesús

# 1. Análisis de Requerimientos

El sistema debe facilitar la interacción natural entre el usuario y el motor de búsqueda semántica mediante una interfaz móvil. A continuación, se detallan los requerimientos divididos por categorías técnicas:

## 1.1 Requerimientos Funcionales

Estos requerimientos definen las funcionalidades que el sistema debe ofrecer:

- **Gestión de consultas:**  
El sistema debe permitir el ingreso de consultas en lenguaje natural tanto mediante texto como a través de voz.
- **Interfaz tipo chatbot:**  
El sistema debe implementar una interfaz de usuario conversacional que permita visualizar las consultas y respuestas en formato de chat.
- **Reconocimiento de voz (STT):**  
El sistema debe integrar mecanismos de captura de audio y conversión automática de voz a texto para el procesamiento de consultas.
- **Síntesis de voz (TTS):**  
El sistema debe reproducir mediante voz las respuestas obtenidas del motor de búsqueda semántica.
- **Comunicación cliente-servidor:**  
El sistema debe establecer comunicación con un servidor remoto mediante una API para enviar consultas y recibir como respuesta los artículos normativos relevantes.
- **Gestión de errores:**  
El sistema debe detectar, manejar y notificar errores relacionados con el reconocimiento de voz y con fallos en la comunicación de red.

## 1.2 Requerimientos No Funcionales (Criterios de Evaluación)

Estos requerimientos definen las cualidades y restricciones que el sistema debe cumplir:

### Eficacia

- El sistema debe alcanzar una tasa de reconocimiento correcto de consultas por voz aceptable, a validar mediante pruebas.
- El sistema debe garantizar una alta tasa de recuperación de información relevante en respuesta a las consultas del usuario

## Eficiencia

- El sistema debe mantener tiempos de respuesta bajos para asegurar una interacción fluida entre el usuario y la aplicación.
- El sistema debe optimizar el uso de recursos del dispositivo (CPU y memoria) para evitar degradación del rendimiento.
- El sistema debe garantizar estabilidad en su ejecución, minimizando la ocurrencia de fallos críticos.

## Usabilidad y Accesibilidad:

- El sistema debe presentar una interfaz intuitiva, clara y consistente que facilite su uso.
- El sistema debe permitir la interacción mediante voz, favoreciendo la accesibilidad y reduciendo la dependencia de entrada textual.

## 1.3 Requerimientos Técnicos (Marco Tecnológico)

Estas son las herramientas y tecnologías definidas para el desarrollo del proyecto:

- **Lenguaje:** Se utilizará Kotlin como lenguaje de programación principal.
- **Plataforma:** Desarrollo nativo para el sistema operativo Android (Android SDK).
- **Interfaz de usuario:** Uso de frameworks modernos como Jetpack Compose o interfaces basadas en XML.
- **Comunicación:** Implementación mediante websocket.
- **Módulos de voz:**
  - **STT:** Uso de servicios nativos de Android (SpeechRecognizer) o soluciones externas.
  - **TTS:** Motor nativo de Android (TextToSpeech).

---

## Propuesta de Diseño Arquitectónico

La arquitectura se organizará en capas para asegurar que el sistema sea modular, tal como se solicita en los criterios de evaluación:

### 1. Capa de Presentación (View Layer)

- **Interfaz:** Implementar la interfaz tipo chatbot usando **Jetpack Compose** o **XML**.

- **Responsabilidad:** Esta capa solo se encarga de mostrar la información y capturar la interacción del usuario. El estado de la UI (por ejemplo, "cargando", "mostrando respuesta", "error") será observado desde el ViewModel.

## 2. Capa de Lógica (ViewModel Layer)

- **Responsabilidad:** Es el cerebro de la pantalla. Aquí manejamos la lógica de la conversación.
- **Funciones:**
  - Gestionar el flujo de datos entre la UI y el repositorio.
  - Invocar los servicios de voz (STT para capturar la pregunta, TTS para reproducir la respuesta).
  - Almacenar el historial de la conversación actual para mantener el contexto del chat.

## 3. Capa de Servicios de Voz (Services Layer)

- **STT (Speech-to-Text):** Utilizarás **SpeechRecognizer** (nativo de Android) para convertir el audio del usuario en texto.
- **TTS (Text-to-Speech):** Utilizarás **TextToSpeech** (nativo de Android) para la síntesis de voz, configurando el motor para que las respuestas se escuchen claras y naturales.

## 4. Capa de Datos (Data/Network Layer)

- **API Client:** Usarás **Retrofit** o **Ktor** para realizar las peticiones HTTP al servidor del proyecto PTAH.
- **Responsabilidad:** Encapsular la lógica de comunicación, manejo de errores de red y serialización de los datos (JSON a objetos Kotlin) para que el resto de la app no tenga que preocuparse por cómo se obtienen los datos del servidor.

# DOCUMENTACIÓN SPRINT 1

## 1. Definición de Arquitectura y Esqueleto Estructural

Durante el Sprint 1, el equipo se enfocó en construir una base sólida y organizada para el proyecto.

Para esto, se eligió una arquitectura MVVM y principios de Clean Architecture, separando el proyecto en distintas capas para facilitar el mantenimiento y el trabajo en equipo. La capa de UI quedó destinada a las pantallas y ViewModels, la capa Domain a la lógica principal de la aplicación y los casos de uso, y la capa Data al manejo de datos externos y conexiones de red.

Además, se configuraron Jetpack Compose para la interfaz, Retrofit para la comunicación con servicios externos y Coroutines de Kotlin para ejecutar tareas de manera asíncrona y mantener una experiencia fluida en la app. Finalmente, se usó el paquete com.uader.ptah como estándar.

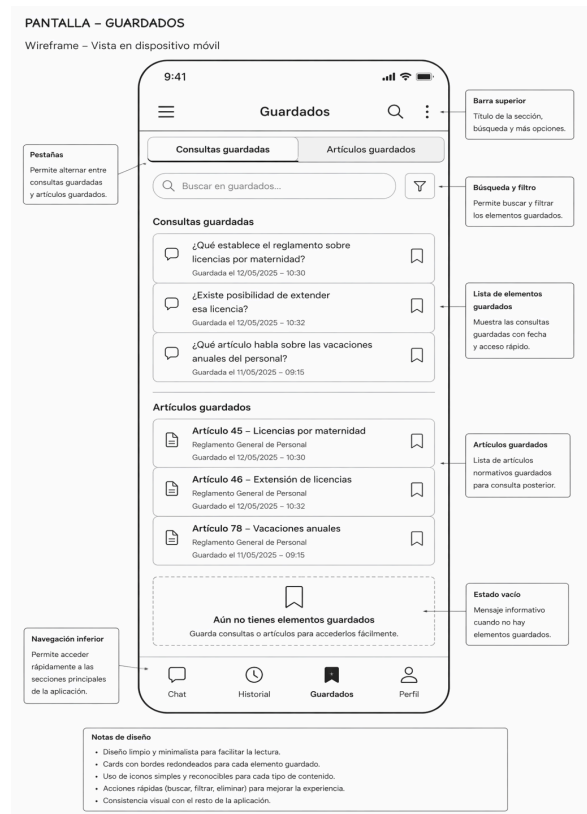
## 2. Diseño y Desarrollo de la Interfaz de Usuario (UI/UX)

En esta etapa el equipo se centró en crear la interfaz con Jetpack Compose, lo que permitió crear una interfaz más dinámica, rápida y organizada ya que usa componentes reutilizables.

También se diseñó el sistema de conversación tipo chatbot, diferenciando visualmente los mensajes del usuario y las respuestas del asistente. Además, se implementaron distintos estados visuales para indicar cuándo la aplicación está escuchando una consulta por voz, procesando información o mostrando una respuesta. Por otro lado, se optimizó la barra de interacción para que el usuario pueda escribir o usar comandos de voz de manera simple y sin interrupciones.

Como resultado, se obtuvo una interfaz funcional, coherente y preparada para integrarse posteriormente con la lógica de negocio y los servicios de red de la aplicación.





### 3. Implementación de Módulos de Procesamiento de Voz (STT y TTS)

Incorporación de las funciones de voz de la aplicación. Para ello, se implementaron los módulos de reconocimiento y síntesis de voz.

Por un lado, se integró el sistema Speech-to-Text (STT) utilizando el motor nativo de Android. Con esto se capta la voz del usuario, se la convierte a texto y luego la procesa. Además, se gestionaron los permisos del micrófono y se añadieron estados visuales para indicar cuándo la aplicación está escuchando.

Por otro lado, se configuró el módulo Text-to-Speech (TTS), encargado de leer en voz alta las respuestas del asistente. Se ajustaron el idioma, la velocidad y el tono de voz.

También se trabajó en la administración de los recursos de audio, asegurando que el micrófono y el sintetizador de voz se detengan automáticamente al cerrar o pausar la aplicación, mejorando el rendimiento y la privacidad del usuario.

Como resultado, la aplicación quedó preparada para mantener una interacción conversacional completa, siendo capaz de escuchar consultas y responder mediante voz de manera fluida.

### 4. Capa de Red y Conectividad (Networking)

Durante este hito del Sprint 1, el equipo trabajó en la capa de red de la aplicación, creando la conexión entre PTAH y el motor encargado del procesamiento. El objetivo fue lograr que las consultas del usuario pudieran enviarse y recibirse de manera rápida, segura y sin afectar la experiencia de uso.

Para esto, se implementó Retrofit como cliente principal de comunicación HTTP y Gson para transformar automáticamente las respuestas JSON del servidor en objetos de Kotlin.

También se definieron las clases de datos necesarias para el intercambio de información entre la app y el servidor, organizando correctamente tanto las consultas realizadas por el usuario como las respuestas obtenidas del sistema. Además, se utilizó Kotlin Coroutines para ejecutar las peticiones de red de forma asíncrona.

Por último, se dejaron preparados los servicios y configuraciones necesarias para integrar el motor semántico, realizando pruebas iniciales de conectividad para asegurar que la arquitectura pueda escalar luego.

Como resultado, la aplicación quedó conectada a servicios externos y preparada para interactuar en tiempo real con sistemas inteligentes de búsqueda y procesamiento de información.

## **5. Gestión de Versiones y Metodología de Trabajo (Git & GitHub)**

El equipo organizó la metodología de trabajo utilizando GitHub para facilitar el desarrollo colaborativo y evitar la pérdida de avances. Gracias a esto, cada integrante pudo trabajar en distintas partes del proyecto de manera ordenada y segura.

Se implementó una estrategia de ramas dentro de un repositorio donde “main” quedó reservada para las versiones estables del proyecto, mientras que “develop” se utilizó para integrar los avances generales uno por uno. Además, cada funcionalidad se desarrolló en ramas independientes creadas por cada integrante según lo que le correspondía hacer en el sprint, permitiendo trabajar en paralelo sin afectar el trabajo de los demás.